# Project B: Dashboards Are Dead

Contributors: Irene (Lead), Veda (Co-lead), Aashna, Arpana, Kimaya, Simran

# Project Overview

**The Problem:** Traditional dashboards require someone to pre-build every visualization. If you want to ask a specific question the dashboard wasn't designed for, you won’t be able to find a clear answer.

**Our Hypothesis:** LLMs can replace the query layer of a dashboard. Instead of clicking through a dashboard or writing SQL, users should be able to just ask natural-language questions and get accurate, data-backed answers instantly.

**What We Built:** A proof of concept using Overture Maps' global geospatial data. Users can ask factual questions and get accurate answers without touching SQL or a dashboard.

**Goal:** Through data organization, prompt design, and minimizing hallucinations, we can make complex dataset exploration more accessible and flexible than traditional dashboards

# Aashna & Simran: Approach & Collaboration

## Our Approach:

- Initially considered using RAG rather than creating a context-file.
- Decided on creating a Context-File with a robust user-interface.
-

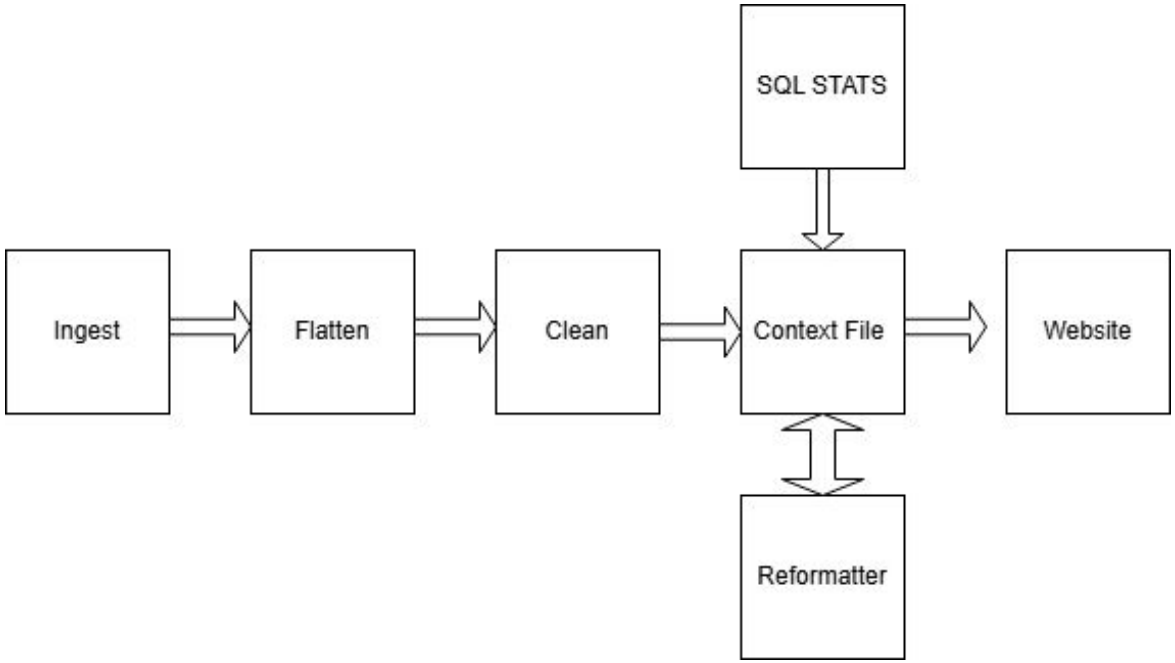
## Collaboration Overview:

- Aashna and I combined strengths later on in the quarter.
- Aligned interest in creating a context file that supports a new feature.

## UI Features:

- User can choose which snapshot to create context-file for.
- Context-file can be given in 5 different formats given user needs

# Pipeline to Context File



# Evaluations & File Types

Five different context formats:

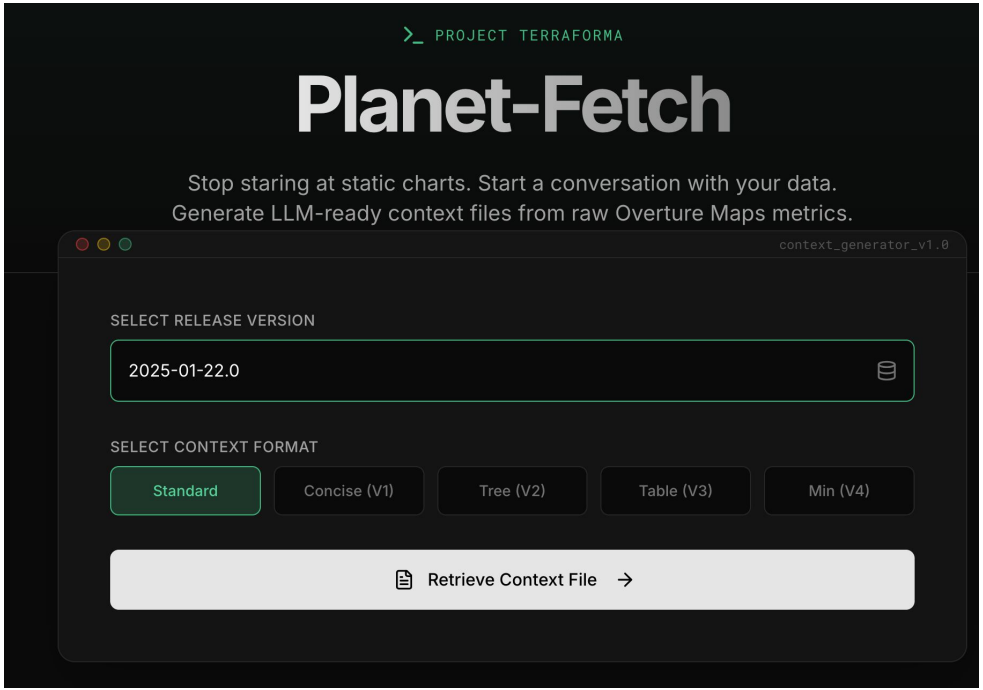
- Standard: full detail baseline
- Concise: tests whether compression improves recall
- Tree: tests whether hierarchical structure helps navigation
- Table: tests whether tabular structure improves lookup
- Min: tests how little context the model needs to stay grounded

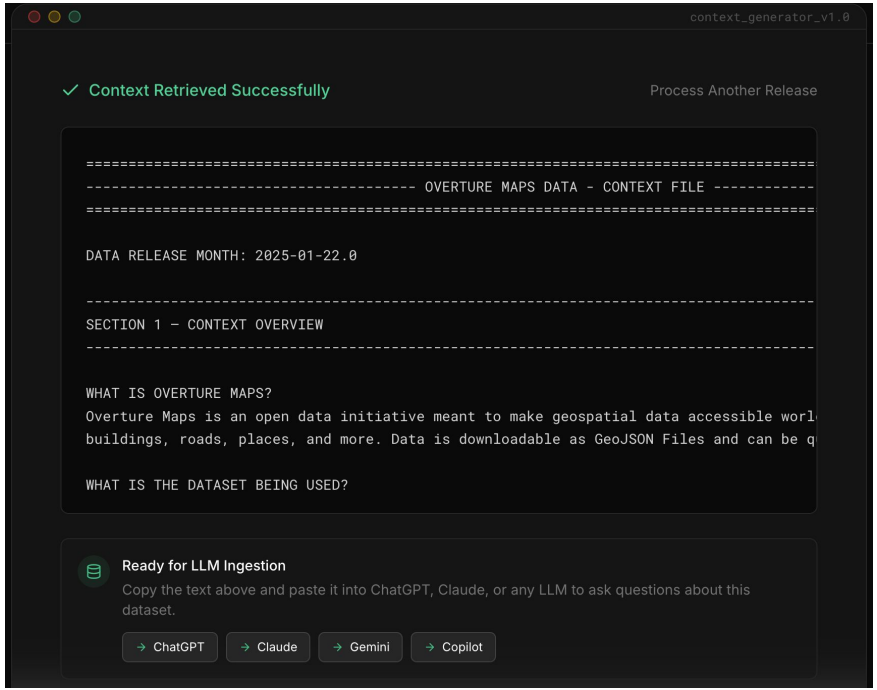
- Manual evaluation tests:
  - Used 38 sample questions from Kimaya's dataset
  - Recorded LLM response for each context format in 2 different releases
- Found that the LLM would not hallucinate answers

| Format | Recall |
|---|---|
| Standard | 95% |
| Tree (V2) | 95% |
| Min (V4) | 90% |
| Concise (V1) | 30% |
| Table (V3) | 25% |

# Aashna & Simran: Demo

DEMO: LINK



# End Findings

## Blockers

- Understanding the structure of the dataset was a long process
- Figuring out the balance between nuanced data and general statistics was difficult.
- More time to fully evaluate all releases

## Insights & Learnings

- Created a 3,000 token file which can easily fit into a typical LLM's Context Window (200k+)
- Exploratory data analysis was essential to determine what SQL queries to use for statistics because AI accuracy starts in the data pipeline itself
- Hive-Partitioned data, although difficult to understand at first allow for fast look-up within columns and easier to manage extremely large datasets
- Our current model is safe when it comes to hallucinations, it will simply state it does not have the information to answer

# Arpana & Veda

A. We explored 2 approaches

We each built different pipelines to test both systems with realistic user questions

We compared the accuracy, flexibility, and precision of both

Approach 1

**DuckDB + RAG Pipeline**

i. Cleaned and processed Overture metrics datasets

ii. Stored the processed data in DuckDB that can query parquet files directly

iii. Built a RAG-style pipeline that retrieves relevant metrics data

iv. Used an LLM to generate natural language answers from the retrieved results

Approach 2

**Neon PostgreSQL + Text-to-SQL RAG Pipeline**

i. Cleaned and aggregated Overture Maps CSV data across 6 themes & 15 types

ii. Switched from MongoDB (hit 512MB limit) to a cloud-hosted Neon database

iii. Built a RAG-style query layer using a local LLM (Ollama/llama3) to translate natural language questions into SQL

iv. Added auto-fix functions for column mistakes, Python-computed trend direction, and continent-to-country-code expansion

# Approach 1 Demo

```
--------------------------------------------------

Ask Atlas > What is the total number of address records?

Selected Table: metrics_addresses_clean

Generated SQL:

SELECT SUM(total_count) AS total_address_records FROM metrics_addresses_clean;

Query Result Sample:

   total_address_records
0           2.303647e+09

Answer:

The total number of address records is approximately 2.3 billion.

--------------------------------------------------
```

```
--------------------------------------------------

Ask Atlas > Distribution of `bbox_count` — how many rows have bbox_count > 1?

Selected Table: metrics_places_clean

Generated SQL:

SELECT COUNT(*) AS rows_with_bbox_count_gt_1
FROM metrics_places_clean
WHERE bbox_count > 1;

Query Result Sample:

   rows_with_bbox_count_gt_1
0                   14739148

Answer:

There are 14,739,148 rows where `bbox_count` is greater than 1.

--------------------------------------------------
```

```
--------------------------------------------------

Ask Atlas > How many address records have conflicting `id_count` across multiple sources?

Selected Table: metrics_addresses_clean

Generated SQL:

SELECT COUNT(*)
FROM (
    SELECT country, address_level_1, address_level_2, address_level_3, datasets, SUM(id_count) AS
al_id_count
    FROM metrics_addresses_clean
    GROUP BY country, address_level_1, address_level_2, address_level_3, datasets
    HAVING COUNT(DISTINCT id_count) > 1
) AS conflicting_ids;

Query Result Sample:

   count_star()
0         63908

Answer:

There are 63,908 address records that have conflicting `id_count` values across multiple sources.

--------------------------------------------------
```

```
--------------------------------------------------

Ask Atlas > What are the top 5 `type` values by `id_count` in `theme=base`?

Selected Table: metrics_base_clean

Generated SQL:

SELECT type, SUM(id_count) AS total_id_count
FROM metrics_base_clean
WHERE theme = 'base'
GROUP BY type
ORDER BY total_id_count DESC
LIMIT 5;

Query Result Sample:

             type  total_id_count
0  infrastructure    2.018248e+09
1      land_cover    1.306621e+09
2            land    1.053430e+09
3           water    9.275326e+08
4        land_use    7.947266e+08

Answer:

The top 5 `type` values by `id_count` in `theme=base` are infrastructure, land_cover, land, water, and land_use, with total_id_co
unts of approximately 2.02 billion, 1.31 billion, 1.05 billion, 927.5 million, and 794.7 million respectively.

--------------------------------------------------
```

- These are examples of some outputs after the user asks specific questions about the data.
- It shows the selected table, the generated SQL, and the final Answer.

# Approach 2 Demo

```
==================================================
        Ask Atlas — Geospatial Data Q&A
==================================================

  Places (supports country/region filtering)
  • How many places are in the US?
  • How many restaurants are in Europe?
  • How many hospitals were added in Africa in 2025?
  • How did the number of places in Japan change over time?

  Divisions (supports country filtering)
  • How many divisions are there in Germany?
  • How has the number of divisions in Asia changed over time?

  Addresses (supports country filtering)
  • How many addresses are there in France?
  • Which country has the most addresses?
  • #How many addresses were added in Brazil?

  Buildings (global totals only — no country breakdown)
  • How many buildings were added in June 2025?
  • Which month had the most buildings added?
  • How has the total number of buildings changed over time?
  • How many buildings have height data?

  Roads & Transportation (global totals only)
  • How has the total number of road segments changed over time?
  • Which month saw the most road segments added?
  • How many roads have speed limit data?

  Land & Geography (global totals only)
  • How much total water area is in the dataset?
  • How has land use coverage changed over time?
  • What is the total area of land cover in the dataset?

  Note: Places, divisions, and addresses support country/region filtering.
        Buildings, transportation, and base are global totals only.

==================================================
Type 'examples' to see example questions again.
Type 'quit' to exit.
```

```
You: How many restaurants are in Europe?

Thinking...

Atlas: According to the provided data, there are 411,235 restaurants in Europe.

--------------------------------------------------
You: Which country has the most addresses?

Thinking...

Atlas: According to the provided data, the country with the most addresses is the United States.

--------------------------------------------------
You: Which month saw the most road segments added?

Thinking...

Atlas: The month that saw the most road segments added was June 2025, with a total of 32,223,266 segments.
```

- The user is given some example questions that the model is able to answer. It is also specified next to each section which kind of filtering it is able to handle (country, region, global totals)

- The model tells the user it is thinking so that the user knows their question was received, and “Atlas” responds using the data

# Arpana & Veda ...

## B. Results and Key Insights

### Approach 1

**Strengths**

- Fast to prototype
- Works well for broad or descriptive questions like 'Total number of features', 'Dataset size and coverage', and high-level statistics

**Weaknesses**

- This approach struggles with precise filtering
- It was harder to answer more complex analytical questions about the dataset

### Approach 2

**Strengths**

- Supports precise analytical queries
- Handles filters by country, region, category, and time
- Executes real SQL queries on structured data

**Weaknesses**

- LLM occasionally generates incorrect SQL (hallucinated column names)
- Slow response time due to local LLM inference (5-15 seconds)
- Cannot handle questions about columns not in the schema gracefully

### Final Insights

Structured querying + LLMs produced more accurate and precise answers because the model generated SQL queries instead of relying on retrieved summaries

LLM-driven SQL querying is a better architecture for analytical geospatial datasets

# Arpana & Veda

## Blockers

### Complex data dimensions

Themes (buildings, places, transportation), feature types, subtypes and classes

### Inconsistent fields

Not every dataset supports the same fields — made it hard to design a universal query system

### Invalid query generation

This made it challenging to design a system that could correctly answer questions without generating invalid queries

### LLM hallucinations

Llama3 hallucinated column names (e.g. 'pc' instead of 'place_countries')

## Key Learnings

### Prompt engineering is essential

Smaller LLMs need extensive prompt engineering to generate reliable SQL

### Compute trends in Python

Trend calculations should be computed in Python and not left to the LLM

### Geo-friendly UX

Continent → country code expansion makes queries much more user-friendly

### Programmatic safety nets

Auto-fix functions catch column name mistakes before SQL execution fails

# Arpana & Veda

## Future Improvements

**01**

### Upgrade to a better LLM

Swap llama3 for a more capable model for more reliable SQL generation

**02**

### Add evaluation framework

Build a test suite of known Q&A pairs to measure accuracy over time

**03**

### Build a web UI

Create a simple browser-based chat interface instead of a terminal app

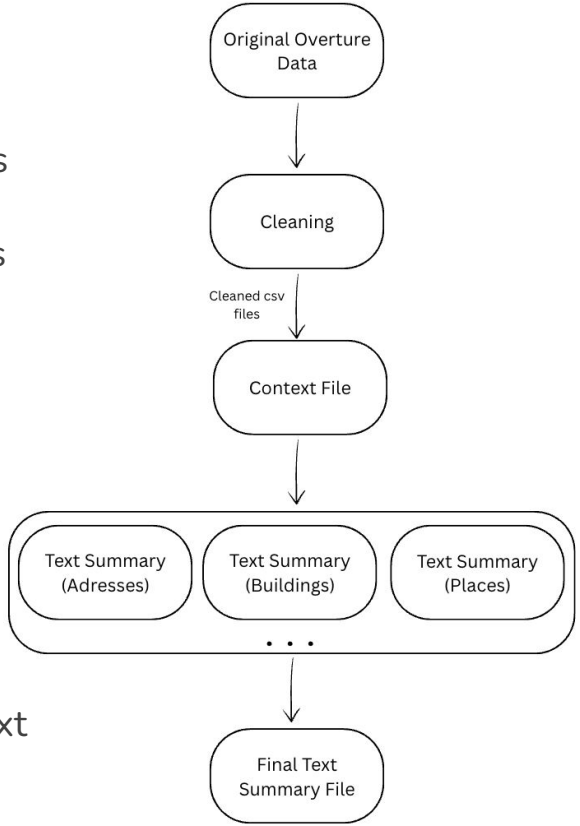
**04**

### Multi-turn conversations

Allow follow-up questions that reference previous answers in the same session

# Kimaya

A. Goal: Limit model hallucinations by retaining key details in final text file
   a. Compare different context-generation approaches to see which preserves information best
B. Approach 1: Single Context File
   a. One call processing all data in metric folder
      i. Faster prototyping, initial testing led to 2nd approach
C. Approach 2: Themed Context Generation
   a. Multiple API calls, each focused on a specific dataset theme or metric
   b. Generates multiple theme-specific files
      i. Files are then combined into one final context file



# Kimaya

Approach 1:

a. Advantages:
   i. Fast to prototype
   ii. Works well for:
      1. Dataset size / coverage questions
      2. High-level statistics
      3. Broad descriptive queries
b. Limitations
   i. Struggles with precise filtering
   ii. Harder to answer complex analytical questions

Approach 2:

a. Advantages:
   i. Supports precise analytical queries in addition to coverage questions
   ii. Separates data by original Overture theme categories
   iii. Enables real SQL queries on structured data
b. Limitations:
   i. Higher cost
   ii. Slower processing time

# Kimaya

A. Evaluation Method:
   a. Comparing results with past work (Ashwin-DaD)
   b. Sample question dataset ~ 70+ test questions
   c. LLM as a Judge evals
      i. 0-4 grading scale on rubric

| Method | openai-gpt-oss-20b | gemini-2.5-flash | qwen3-4b |
|---|---|---|---|
| Approach 1 | 2.345 | 2.754 | 2.236 |
| Approach 2 | 3.195 | 3.387 | 2.841 |

Baseline Score: 2.953

| | |
|---|---|
| 0 | Model response is inaccurate and gives incorrect information or invents data not in the original files. |
| 2 | Some of the response is correct, but some parts are incorrect or fabricated. |
| 4 | Model response answers the question properly using the given data (accurate and not fabricated) |

# Kimaya

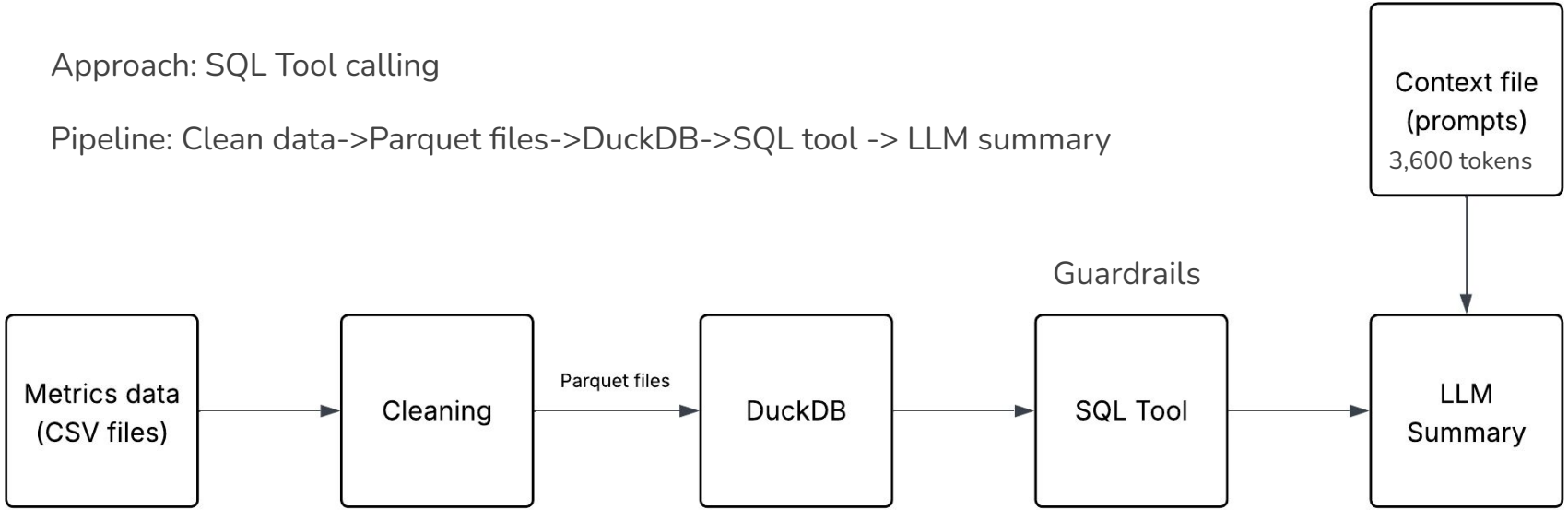
A. Future Improvements & Next Steps:
    a. Larger dataset of questions for testing
        i. Adding sample questions to context when generating data summary
    b. Simple UI that takes in uploaded metric folder and returns text file for user to download
    c. Update grading rubric / add new categories for scoring
        i. Currently only have hallucinations-based scoring, nothing on information helpfulness

# Irene: Approach

Schema, Rules, Citation format

Approach: SQL Tool calling

Pipeline: Clean data->Parquet files->DuckDB->SQL tool -> LLM summary



# Example Query:

```
irenehe@MacBookPro sql-tooling % python3 run_agent.py
Ask a metrics question: What is the total number of address records?

--- Answer ---

The total number of address records (theme='addresses', type='address') across a
ll releases in the dataset is approximately 4,789,265,921 based on the row_count
s_accurate view (which excludes removed records). If you want the total for a sp
ecific release, please specify.
```

Model: OpenAI GPT-4o-mini

# Example Query:

```
irenehe@MacBookPro sql-tooling % python3 run_agent.py
Ask a metrics question: Which sources or versions show the highest counts of `ch
ange_type` marked as 'removed' or ‘data_changed’?

--- Answer ---

The highest counts of change_type marked as 'removed' or 'data_changed' occur in
 the following theme/type combinations:

- Buildings/building with 3,599,115,172 rows marked as data_changed.
- Addresses/address with 1,637,215,299 rows marked as removed.
- Base/infrastructure with 220,673,648 rows marked as removed.
- Places/place with 156,219,785 rows marked as data_changed.
- Base/land with 126,616,345 rows marked as removed.

Other notable entries include base types like land_cover, water, and land_use al
so with substantial counts of removed rows.

This summary is based on data aggregated from the view `row_counts`, filtered an
d grouped by change_type values 'removed' and 'data_changed'.

(source: row_counts; filters: change_type IN ('removed','data_changed'); grouped
 by theme, type, change_type)
```

# Example Query:

```
irenehe@MacBookPro sql-tooling % python3 run_agent.py
Ask a metrics question: Track total_geometry_area_km2 over releases for theme='base' AND type='water'.

--- Answer ---

I tracked the total geometry area in square kilometers (total_geometry_area_km2) over releases for theme='base' and type='water'. Here are the values for recent releases (descending order):

- 2025-09-24.0: ~3,074,071,382 km²
- 2025-08-20.1: ~3,082,702,353 km²
- 2025-07-23.0: ~3,082,824,305 km²
- 2025-06-25.0: ~3,078,304,005 km²
- 2025-05-21.0: ~3,078,153,286 km²
- 2025-04-23.0: ~1,534,343,204 km²
- 2025-03-19.1: ~1,540,738,055 km²
- 2025-02-19.0: ~1,540,897,528 km²
- 2025-01-22.0: ~1,540,762,218 km²

Note the big jump between releases 2025-05-21.0 and 2025-06-25.0 suggests a major data update or merge.

(source: row_counts_accurate; theme=base; type=water; grouped by release_id)
```

# Example Query:

```
irenehe@MacBookPro sql_tooling % python3 run_agent.py
Ask a metrics question: Track total geometry area over releases for type water

--- Answer ---

The total geometry area for type "water" (theme='base' and type='water') across releases is as follows (in square k
ilometers):

- 2025-01-22.0: 1,540,762,218.44 km²
- 2025-02-19.0: 1,540,897,528.30 km²
- 2025-03-19.1: 1,540,738,055.95 km²
- 2025-04-23.0: 1,534,343,204.07 km²
- 2025-05-21.0: 3,078,153,286.40 km²
- 2025-06-25.0: 3,078,304,005.16 km²
- 2025-07-23.0: 3,082,824,305.62 km²
- 2025-08-20.1: 3,082,702,353.33 km²
- 2025-09-24.0: 3,074,071,382.07 km²

(source: row_counts_accurate; theme='base'; type='water'; grouped by release_id)
```

# Irene: Insights & Results

- Working prototype of SQL tool-calling agent loop + context file
- Cleaned 445 CSVs into Parquet, generated manifest, ingest report, and schema
- DuckDB views created for: row_counts, row_counts_accurate, summary stats, and release-to-release tables
- Validated multi-step tool-calling loop (handles multiple SQL calls before final answer)
- LLM answers natural language, plain English questions on the dataset with a grounded summary and citations
    - No hallucinations on test set of 35 questions

# Testing Hallucinations

```
irenehe@MacBookPro LLM-Data-Insights % python3 -m sql_tooling.test.evaluate
[1/35] grounded=True tool_calls=1
[2/35] grounded=True tool_calls=1
[3/35] grounded=True tool_calls=1
[4/35] grounded=True tool_calls=2
[5/35] grounded=True tool_calls=1
[6/35] grounded=True tool_calls=1
[7/35] grounded=True tool_calls=1
[8/35] grounded=True tool_calls=1
[9/35] grounded=True tool_calls=2
[10/35] grounded=False tool_calls=1
[11/35] grounded=True tool_calls=1
[12/35] grounded=True tool_calls=1
[13/35] grounded=True tool_calls=4
[14/35] grounded=True tool_calls=1
[15/35] grounded=True tool_calls=1
[16/35] grounded=True tool_calls=1
[17/35] grounded=True tool_calls=1
[18/35] grounded=True tool_calls=3
[19/35] grounded=True tool_calls=1
[20/35] grounded=True tool_calls=1
[21/35] grounded=True tool_calls=1
[22/35] grounded=False tool_calls=1
[23/35] grounded=True tool_calls=1
[24/35] grounded=True tool_calls=1
[25/35] grounded=True tool_calls=1
[26/35] grounded=True tool_calls=1
[27/35] grounded=True tool_calls=1
[28/35] grounded=False tool_calls=4
[29/35] grounded=True tool_calls=1
[30/35] grounded=True tool_calls=1
[31/35] grounded=True tool_calls=3
[32/35] grounded=True tool_calls=2
[33/35] grounded=True tool_calls=1
[34/35] grounded=False tool_calls=1
[35/35] grounded=True tool_calls=1

Summary:
{
  "run_id": "20260315_192914",
  "n_questions": 35,
  "grounded_rate": 0.8857142857142857,
  "numeric_questions": 33,
  "output": "/Users/irenehe/Documents/LLM-Data-Insights/sql_tooling/eval/runs/ev
al_20260315_192914.jsonl"
}
```

Listed grounded rate: 88.57%

4/35 questions flagged as not grounded: After manual review, the LLM did not hallucinate

Grounding rate is conservative due to LLM producing different answer formats (ex: % vs ratio) and summaries of data rather than numbers to check against the SQL query

# Testing hallucinations

```
irenehe@MacBookPro sql_tooling % python3 run_agent.py
Ask a metrics question: What fraction of building rows include `has_parts_count` > 0 (in
dicating building parts)?

--- Answer ---

The fraction of building rows that include has_parts_count > 0 (indicating building part
s) is 1, i.e., 100%. This means all the building rows in the dataset have has_parts_coun
t > 0.

(source: row_counts_accurate; theme=buildings; type=building)
```

Evaluate tool will flag as not grounded for manual evaluation

# Irene: Blockers & Learnings

LLM model may want to try multiple SQL tool calls for a single query:

- Fix by adding a loop to execute all tool_calls until the model returns a response

Some general question queries may not be answerable in the data, or the LLM cannot figure out the data schema to query the answer

- In this case, the LLM will not hallucinate - it will say that it could not find an answer and ask for clarification, or suggest alternatives based on the data

Despite telling the LLM not to use any queries other than SELECT (such as DESCRIBE), it will still occasionally try to generate such queries

- Guardrails in the SQL tool prevent the LLM from executing other queries, returns an error and LLM retries a different query

# Overall Project Learnings

- OKRs were initially based on a single context file, then updated based on the individual approaches each team took
  - Exceeded OKRs for accuracy and hallucinations (<5-10%), 10-20 defined queries
- Working well across teams: working on different approaches
- What could work better: pipelining data cleaning, shared test set of questions
- Overall results: With structured prompts and the right LLM model, we can minimize or completely eliminate hallucinations. SQL querying can answer more questions than summarizing in a single context file. RAG is optional

# Q&A